

for-each ループと VEX — 同じことをやって比べる

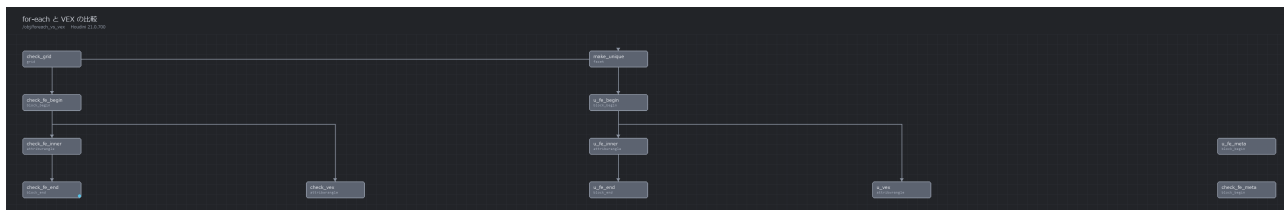
2026-09-13 | Houdini 21.0.700 | /obj/foreach_vs_vex | ノード 12 / 接続 9

何を作ったか

「まとめて処理できないときは for-each」とよく言われるが、VEX でも書ける場面でどれだけ差が出るのかは測らないと分からない。そこで、面ごとに違う高さへ動かすという同じ処理を2通りで作り、結果と速さを比べた。

ところが最初は結果が一致しなかった。その理由を追いかけた結果、この2つは「速い遅い」で選ぶものではなく、そもそも別のことをしているのだと分かった。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
check_grid	grid	cols = 8 rows = 8
check_fe_begin	block_begin	blockpath = ../check_fe_end method = 1
check_fe_meta	block_begin	blockpath = ../check_fe_end method = 2
check_fe_inner	attribwrangle	snippet = @P.y += detail("op:../check_fe_meta", "iteration", 0) * 0.001;
check_fe_end	block_end	blockpath = ../check_fe_begin class = 0 itermethod = 1 method = 1 templatepath = ../check_fe_begin
check_vex	attribwrangle	class = 1 snippet = int pts[] = primpoints(0, @primnum); foreach (int pt; pts) { vector p = point(0, "P", pt); p.y += @primnum * ...
make_unique	facet	unique = 1
u_fe_begin	block_begin	blockpath = ../u_fe_end method = 1
u_fe_meta	block_begin	blockpath = ../u_fe_end method = 2
u_fe_inner	attribwrangle	snippet = @P.y += detail("op:../u_fe_meta", "iteration", 0) * 0.001;
u_fe_end	block_end	blockpath = ../u_fe_begin class = 0 itermethod = 1 method = 1 templatepath = ../u_fe_begin

分割数	プリミティブ数	for-each (ミリ秒)	VEX (ミリ秒)	何倍
32	961	53.55	1.414	37.9倍
64	3,969	191.87	2.233	85.9倍

結果が一致しなかった理由を追う

同じ処理のはずなのに、49面中48面で高さが違った。しかも差は全部 0.004 で一定。ここから3段階で原因を絞り込んだ。

調べたこと	結果	分かったこと
出力の点数を比べる	元 64点 → for-each 196点 / VEX 64点	for-each は面を1枚ずつ取り出すので、隣と共有していた点が複製される
VEX を3回走らせて結果を比べる	3回とも完全に同じ	複数の面が同じ点に書き込む競合ではあるが、順序は決まっていて再現する
点の共有をほどいてから比べる	完全に一致	食い違いの原因は点の共有。ほどけば2つは同じ結果になる

わかったこと

- ・ for-each は Block Begin (method = Extract Piece or Point) と Block End で処理を挟んで作る。Block End の Template Path が空だと「Missing template geometry」で止まる。何を何回繰り返すかはここから決まる。
- ・ 何回目かを知るには、メタデータ用の Block Begin (method = Fetch Metadata) をもう1つ置き、そのディテールアトリビュート iteration を読む。numiterations で総回数も取れる。
- ・ 取り出した中では @primnum が常に 0 になる。1個だけを切り出した状態で処理しているため。「何番目の面か」を使いたいならメタデータを読むしかない。
- ・ detail() のパスには op: が要る。detail("../メタデータノード", "iteration", 0) と書くとファイルパスとして扱われ、エラーも警告も出さずに 0 を返す。ループが動いていないように見えるので原因が分かりにくい。detail("op:../メタデータノード", ...) と書けば動く。実際にこれで1時間相当を溶かした。
- ・ for-each は面の数に比例して遅くなる。49面で11.79ミリ秒、3,969面で191.87ミリ秒。1面あたりおよそ0.05ミリ秒で、面の数がそのまま効く。
- ・ VEX はほとんど変わらない。49面で1.36ミリ秒、3,969面で2.23ミリ秒。実験009で測った「ノード1つあたり約1.8ミリ秒の固定費」がそのまま出ており、処理そのものの時間はまだ見えていない水準。
- ・ その結果、差は面の数とともに広がる。49面で 8.7倍、3,969面で 85.9倍。面が増えるほど for-each を選ぶ代償が大きくなる。
- ・ ただしこの2つは単純に置き換えられるものではない。for-each は面を1枚ずつ取り出すので、隣と共有していた点が複製される (64点 → 196点)。これは副作用ではなく、面ごとに独立して動かすために必要なこと。点を共有したままでは、隣り合う面が同じ点を別の場所へ動かそうとして取り合いになる。
- ・ その取り合いは競合ではあるが、結果は毎回同じだった (3回とも一致)。書き込む順序が決まっているため、実行のたびに変わることはない。最初は並列処理による非決定性を疑ったが、それは外れた。
- ・ 点の共有をほどいてから (facet の Unique Points で 64点 → 196点にしてから) 比べると、for-each と VEX の結果は完全に一致した。つまり食い違いの原因は処理内容ではなく、点が共有されているかどうか

かだけだった。

- ・実用的な結論。面ごとに独立した変形をしたいなら、まず点の共有をほどこ。ほどいてしまえば VEX で書ける場面が増え、数十倍速い。for-each が必要なのは、1回分の処理が VEX 1つでは書けないほど複雑なとき（中で複数のノードを通したいときなど）。

- ・測り方についても1つ。最初は面の中心を (x, y, z) でソートして比べようとしたが、比べたい量である y をソートの鍵に含めてしまったため、対応がずれて必ず食い違った。比べたい量は鍵にしない。水平位置 (x, z) を鍵にして y を値にしたら正しく比較できた。

次にやること

- ・facet の Unique Points を挟むことで、for-each が必要だった処理をどこまで VEX に置き換えられるかを試す
- ・for-each の中で複数ノードを通す場合（VEX 1つでは書けない場合）の実例を作る
- ・polyextrude と boolean に進む